

Module 9: Capstone Project — Student Records & Grading System

Introduction to C++ Programming — A Beginner's Course

Module Overview

The capstone is deliberately staged into three commits, mirroring how real software actually gets built — incrementally, with a working, committed version at every step, rather than as one large uninterrupted build. Your role shifts here from lecturing new material (there is none) to supervising, unblocking, and code-reviewing. Treat this module as a studio/lab session, not a lecture.

Suggested Duration: 2–3 sessions of 60–90 minutes each, or equivalent lab/studio time

Learning Objectives

By the end of this module, students will be able to:

- Design and implement two collaborating classes (Student, Classroom) from a written specification.
- Apply every major concept from Modules 1–8 in a single cohesive program.
- Practice staged, incremental development with a meaningful git commit at each stage.
- Handle a not-found case gracefully (a search that may fail) rather than assuming success.
- Present and explain a completed program confidently, including justifying design choices.

Prerequisites

- All of Modules 1–8. This module assumes fluency, not just familiarity, with every prior concept.

Materials & Instructor Prep

- 09-capstone-project/README.md and starter.cpp available to every student.
- A rubric or checklist tied explicitly to the three stages, so partial credit / partial completion is easy to assess fairly.

Key Vocabulary

Term	Definition
Incremental development	Building a program in small, working, separately-committed stages rather than all at once.
Pointer (brief, practical introduction)	A variable holding the memory address of another variable; used here only to represent 'found this student, or found nothing' via nullptr.
nullptr	A special pointer value meaning 'points to nothing' — used to represent a failed search.

Lesson Outline & Instructor Talking Points

Session 1 — Stage 1: The Student class (concept map + build)

- Open with the concept map table from the capstone README, explicitly naming which module each requirement traces back to. This reframes the capstone as assembly, not new material — a genuinely reassuring framing at this point in the course.
- Have students design addScore's validation (0–100 range) and average()'s divide-by-zero guard themselves before checking the spec — both are direct callbacks to earlier lessons (Module 3 input handling; Module 6 return values).
- Require the Stage 1 commit before ending the session: 'Capstone stage 1: Student class'.

Instructor note: Walk the room actively during build time. The most common Stage 1 stumble is forgetting the average() divide-by-zero guard — this is a good moment to remind students to test with a freshly-created Student that has zero scores.

Session 2 — Stage 2: Classroom + the menu loop

- Introduce Student* findStudent(...) and nullptr as a brief, scoped exception to 'we haven't covered pointers' — frame it honestly: 'This is your first look at a pointer. You don't need to master pointers generally today — just this one pattern: a function that might not find anything needs a way to say so, and a reference alone can't represent 'nothing.''
- Build the do-while + switch menu loop live as a class, since this is a direct, low-risk reuse of Module 4/8 patterns — spend the saved time on the two class designs instead.
- Require the Stage 2 commit: 'Capstone stage 2: Classroom class + menu'.

Instructor note: Expect the pointer introduction to cause the same kind of productive confusion recursion did in Module 7 — that's expected. Reassure students this is a narrow, practical exception, not a sign they need to learn pointers broadly right now.

Session 3 — Stage 3: Polish, validation, and presentation

- Have each student (or pair) pick at least 2 of the polish options: input validation on a not-found name, a simple ranking sort, or the recursive search stretch goal (a direct, deliberate callback to Module 7 — consider requiring this one specifically for strong groups).
- Close with short (2–3 minute) informal presentations: each student demos their program and explains one design decision they made and why.
- Require the final commit: 'Capstone stage 3: polish and validation'.

Instructor note: *The presentation step matters more than it looks — articulating a design choice out loud is a genuinely different skill from writing the code, and it's the last chance in the course to build it.*

Common Student Misconceptions

Misconception	How to Address It
"I should build the whole thing in one sitting, then commit once at the end."	Explicitly require the three separate commits — this is as much a habit-reinforcement exercise (from Module 0) as a coding one.
"findStudent returning nullptr means something went wrong / crashed."	nullptr here is an expected, valid, and meaningful result — 'not found' — not an error state; the caller is responsible for checking it before use.
"Since we haven't formally covered pointers, I shouldn't need to touch Student*."	Reassure directly: this is one narrow, practical pattern, provided in the starter file — full pointer mechanics are explicitly named as a 'next step beyond this course,' not an expectation here.

Exercises & Module Project

The capstone itself IS the assignment — no separate exercises. See 09-capstone-project/README.md for the full three-stage specification and 09-capstone-project/starter.cpp for the scaffolded skeleton.

Facilitation guidance: Grade holistically against the concept map, not just against a single 'correct' output — value clean function/method decomposition and appropriate use of encapsulation as much as raw correctness at this stage.

Discussion & Comprehension Check Questions

1. Which class — Student or Classroom — was responsible for computing the class average, and why did you design it that way?

Guidance: There's a reasonable design debate here (Classroom aggregating each Student's own average() vs. Classroom reaching into raw score data) — use it to surface that OOP design involves real judgment calls, not one fixed right answer.

2. What would you need to change in your design if a student could be enrolled in more than one classroom?

Guidance: Not expected to be solved — the goal is recognizing that today's one-classroom assumption is a simplification, and noticing where it would need to change.

Assessment Ideas

- Use the three staged commits themselves as a lightweight process rubric (did each stage actually get committed separately, with a reasonable message) alongside a functional correctness check.
- The closing presentation doubles as an informal oral assessment — listen for whether a student can justify a design choice, not just describe what the code does.

Answer Key Notes

There is no single solutions file for the capstone by design — it's an open-ended synthesis project. Use the concept map and the three-stage checklist in 09-capstone-project/README.md as your grading anchor instead of a fixed reference implementation.

Wrap-Up & Bridge to Next Module

Close the course, not just the module. Point at the roadmap one final time and name what's now true that wasn't at Module 0: every student has a real, multi-stage, committed project on their own GitHub profile. Name the explicit next steps from the capstone README (inheritance/polymorphism, deeper pointers, file I/O, the standard library) as an honest, motivating map of what's next — not a vague 'keep learning' send-off.

Instructor Tips

- Protect studio/lab time above all else in this module — your value here is walking the room, unblocking stuck students, and asking design questions, not lecturing.
- Consider a short live demo of your own reference solution's Stage 3 recursive search, hand-traced on the board exactly like Module 7's factorial — it's a genuinely satisfying full-circle moment for students who found recursion hard the first time.